Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

06

MODULE 06

Git and GitHub in VS Code

Do everything you learned with buttons: stage, commit, push, branch and sync.

MODULE 06 · LESSON 6.1

VS Code Source Control

لوحة التحكم بالمصدر في VS Code

Learning objectives

- Open the Source Control view.
- Read the status letters beside files.
- Stage, stage all and discard changes.
- Write a commit message and commit.

Prerequisites

Module 4 and the project open in VS Code.

Introduction

VS Code has Git built in. The Source Control view gives you buttons for the commands you already know.

Explanation

Open it. Click the Source Control icon in the left bar (it looks like a branch) or press `Ctrl+Shift+G`. A number badge shows how many files changed.

Changed files. Files appear under **Changes**. A letter on the right shows the status: **M** modified, **U** untracked, **A** added (staged new file), **D** deleted. Click a file to open a side-by-side comparison with the last commit.

Stage Changes. Hover over a file and click **+**. This equals `git add file`. The file moves to **Staged Changes**.

Stage All Changes. Click **+** on the Changes heading. This equals `git add .`. Check the list first.

Unstage. Click **–** on a staged file. This equals `git restore --staged file`.

Discard Changes. The arrow-shaped button beside a file throws away your edits in that file. **Warning: this cannot be undone.** VS Code asks you to confirm.

Commit. Type a message in the box and click **Commit**. This equals `git commit -m "message"`. If nothing is staged, VS Code may offer to stage everything and

commit. Prefer to stage deliberately.

Real-world analogy

> Source Control is the dashboard of a car: the same engine (Git), but with buttons and gauges instead of typed instructions.

Diagram

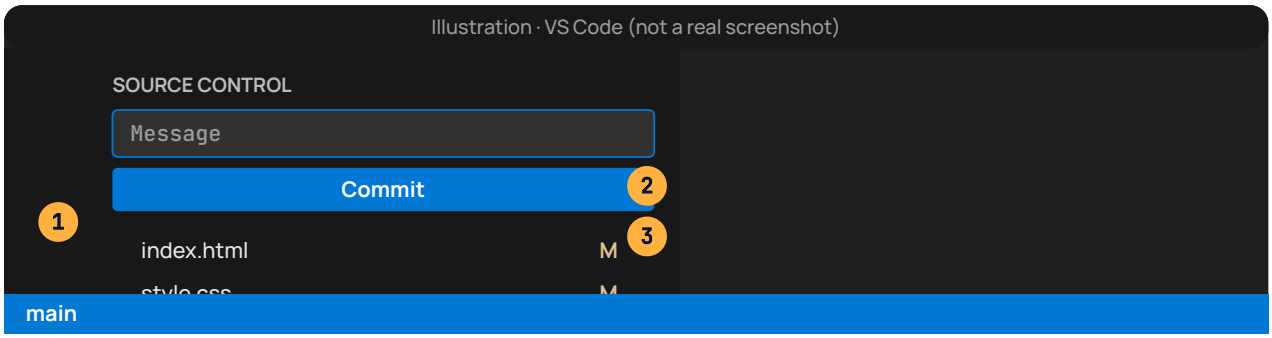


Figure 1 · (1) Source Control icon, (2) Stage a file, (3) Untracked file

Commands

Equivalents: **+** = `git add`; **−** = `git restore --staged`; **Commit** = `git commit -m`. Figure 1 shows the panel with three changed files.

Common mistakes and solutions

Clicking Discard by accident.	Discard deletes your edits. Read the confirmation carefully.
Committing with an empty message.	VS Code requires a message. Write a clear one.
Not seeing the Git view.	Open the project folder itself (File → Open Folder), not a parent folder.

Practical exercise

> Edit `index.html`, open Source Control, click the file to see the comparison, stage it with **+**, then unstage it with **−**.

Expected result

The file moves between Changes and Staged Changes, and the badge number updates.

Knowledge check

1. What does M mean beside a file?

- ○ Moved
- ○ Modified
- ○ Merged

2. Which button equals git add .?

- ○ + on the Changes heading
- ○ The Commit button
- ○ Discard

3. Can Discard Changes be undone?

- ○ Only on Fridays
- ○ No
- ○ Yes, always

Answers

- Modified — M marks a modified tracked file.
- + on the Changes heading — It stages every change.
- No — It permanently removes unsaved-to-Git edits.

Summary

Source Control shows changed files with status letters and gives buttons to stage, unstage, discard and commit.

Key vocabulary

Source Control	VS Code's built-in Git view
Staged Changes	Files ready for the next commit
Discard Changes	Delete your edits in a file permanently

Next lesson

> **Lesson 6.2 — Commit and push without the terminal.** A full workflow, step by step.

MODULE 06 · LESSON 6.2

Commit and push without the terminal

الحفظ والرفع بدون الطرفية

Learning objectives

- Complete edit → stage → commit → push in VS Code.
- Review differences before committing.
- Verify the result on GitHub.

Prerequisites

Lesson 6.1 and a project already pushed (Module 5).

Introduction

Now you do the whole daily workflow with the mouse.

Explanation

1. Open the project. File → Open Folder → `my-first-project`.

2. Edit `index.html`. Add `<p>Learning Git with Programming Worlds.</p>` under the heading. Save with `Ctrl+S`.

3. Open Source Control (`Ctrl+Shift+G`).

4. Review. Click `index.html`. The right side shows the new line in green.

5. Stage the file with **+**.

6. Write a message: `Add intro paragraph`.

7. Commit. Click **Commit**.

8. Push. A **Sync Changes** button appears (with arrows and a number). Click it. Sync pulls remote changes, then pushes yours.

9. Verify. Open your repository on GitHub and refresh. The new commit and the changed file are there.

If the button says **Publish Branch**, the branch has never been pushed. Click it once.

Real-world analogy

> It is the same parcel service as before, just booked online instead of by phone.

Diagram

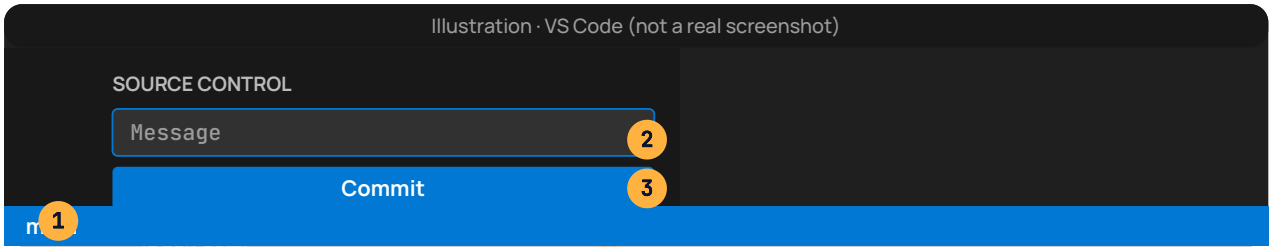


Figure 1 · (1) Source Control, (2) message box, (3) Commit

Commands

Terminal equivalent of steps 5 to 8: `git add index.html`, `git commit -m "Add intro paragraph"`, `git push`.

Common mistakes and solutions

Forgetting to save the file.	An unsaved file shows a dot and not in Source Control.
Sync asks to “pull and push”.	That is normal. Confirm.
Sign-in prompt appears.	Sign in to GitHub in the browser window and approve.

Practical exercise

> Do the nine steps with your own paragraph and message.

Expected result

GitHub shows the commit “Add intro paragraph” and the new line in `index.html`.

Knowledge check

1. Which VS Code button uploads your commit?

- ○ Stage All
- ○ Sync Changes
- ○ Discard

2. What does Publish Branch mean?

- ○ The branch is not on GitHub yet
- ○ The branch is deleted
- ○ The branch is merged

3. Where do you verify the push?

- ○ In Settings
- ○ On GitHub
- ○ In Notepad

Answers

- Sync Changes — Sync pulls then pushes.
- The branch is not on GitHub yet — It pushes a new branch.
- On GitHub — Refresh the repository page.

Summary

You edited, reviewed, staged, committed and synced from VS Code, then verified on GitHub.

Key vocabulary

Sync Changes	Pull then push
Publish Branch	Push a branch for the first time

Next lesson

Lesson 6.3 — Branches and history in VS Code.

MODULE 06 · LESSON 6.3

Branches and history in VS Code

الفروع والتاريخ في VS Code

Learning objectives

- Read the branch indicator.
- Create and switch branches.
- View file history.
- Know where conflicts and sign-in live.
- Tell built-in Git from GitLens.

Prerequisites

Lesson 6.2.

Introduction

Branches are covered in depth in Module 7. Here you only learn where the controls live.

Explanation

Branch indicator. The bottom-left of the window shows the current branch, such as `main`.

Switch or create. Click it. A list opens at the top: choose a branch to switch, or choose **Create new branch…** and type a name such as `feature/homepage`.

History. In the Explorer view, open the **Timeline** panel at the bottom to see the commits that changed the open file. Click one to compare. Recent versions also offer a graph of commits inside Source Control.

Compare changes. Click any changed file in Source Control to see old and new side by side.

Conflicts. If a merge conflicts, files appear under **Merge Changes**. Open one and use **Resolve in Merge Editor**. Module 7 teaches this.

Sync Changes pulls then pushes in one click.

GitHub sign-in. The Accounts icon at the bottom of the left bar shows who is signed in and lets you sign in or out.

GitLens is an optional extension that adds blame, richer history and more. It is not needed for this course. Everything above is built in.

Real-world analogy

> The status bar is a car's road sign: it always tells you which road (branch) you are on.

Diagram

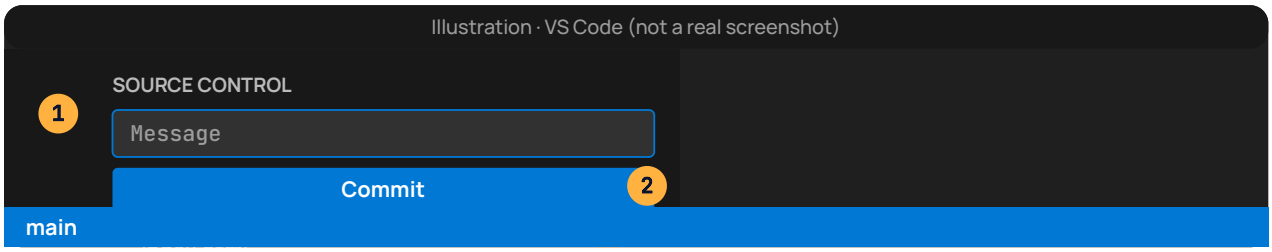


Figure 1 · (1) Branch indicator, (2) Changes

Commands

Equivalents: `git branch` shows the branch; `git switch -c feature/homepage` creates one; `git switch main` switches.

Common mistakes and solutions

Editing on main by accident.	Look at the bottom-left before you start.
Switch blocked by changes.	Commit your changes first.
Installing GitLens and getting lost.	Learn the built-in tools first.

Practical exercise

> Create a branch `feature/homepage` from the status bar, then switch back to `main`.

Expected result

The status bar shows `feature/homepage`, then `main` again.

Knowledge check

1. Where do you see the current branch?

- ○ Settings
- ○ Bottom-left status bar
- ○ The README

2. Is GitLens required?

- ○ No, it is optional
- ○ Yes
- ○ Only for pushing

3. Where do you sign in to GitHub?

- ○ The Timeline
- ○ The Accounts icon
- ○ The Terminal menu

Answers

- Bottom-left status bar — It is shown in the status bar.
- No, it is optional — Git is built into VS Code.
- The Accounts icon — Accounts manages sign-in.

Summary

Branch controls live in the status bar, history in Timeline, conflicts in the Merge Editor, and sign-in under Accounts.

Key vocabulary

Status bar	The bar at the bottom of VS Code
Timeline	Per-file history panel
GitLens	Optional extension for extra Git features

Next lesson

Module 7 — Branches and merging.

MODULE 06

Practical lab 6 · A full workflow using only VS Code

1. Open `my-first-project` in VS Code.
2. Add a paragraph to `index.html` and save.
3. Review the change in Source Control.
4. Stage only `index.html`, commit with `Add intro paragraph`, and click Sync Changes.
5. Check GitHub.

Check	Pass if…
No terminal	You used only buttons
Commit	Message starts with a verb
GitHub	New commit visible online

Evaluation: 3 of 3 = pass.

Module 6 quiz (pass mark: 70%)

1. Which icon opens Source Control?

- ○ The magnifier
- ○ The gear
- ○ The branch-shaped icon

2. What does U mean?

- ○ Undo
- ○ Untracked
- ○ Updated

3. What does Sync Changes do?

- ○ Pulls then pushes
- ○ Deletes files
- ○ Creates a repo

4. Where is the current branch shown?

- ○ Dashboard
- ○ Status bar
- ○ README

5. Is GitLens required?

- ○ Yes
- ○ Only on Windows
- ○ No

Answers

- The branch-shaped icon — Or press Ctrl+Shift+G.
- Untracked — New file Git does not track.
- Pulls then pushes — It synchronizes both ways.
- Status bar — Bottom-left.
- No — Optional extension.